新人研修 ・ 配布用

フロントエンド研修

バックエンドエンジニアのための
HTML / JavaScript / TypeScript / React 入門

第1部　HTML/CSS・JavaScript

第2部　テンプレートエンジン

第3部　Node.js・TypeScript

第4部　React・Next.js

はじめに

この研修の進め方：4部構成

部	テーマ	画面の作り方	作るもの
第1部	HTML/CSS、JavaScript	ブラウザで、表示済みの画面に動きを付ける	タスク一覧のHTMLと、JavaScriptによる追加・完了切替
第2部	テンプレートエンジン	サーバーがHTMLを組み立てる	Spring Boot + Thymeleaf のタスク一覧画面
第3部	Node.js、TypeScript	画面は作らず、JSONを返すAPIを作る	タスクAPIと、そのTypeScript化
第4部	React、Next.js	ブラウザ、またはサーバーとブラウザの両方で組み立てる	第3部のAPIを使うタスク画面

4部とも同じ「タスク管理アプリ」を作る。説明 → やってみよう → 解答、のくり返しで進める

はじめに

この研修のゴール

**「画面側で何が起きているか」を説明でき、
簡単な画面なら自分で作れる**

立ち位置

バックエンドが軸。フロントエンドは補助的な技術として、フルスタックへ広げる最初の一歩

進め方

短い説明のあと、すぐ手を動かす。「Javaならどう書くか」を必ず対比する

題材

すべての部で同じ「タスク管理アプリ」。同じものを3通りの方式で作り直す

フロントエンドのさまざまな構築方法

構築方法	代表的な技術
静的Webページ	HTML・CSS、nginx
静的サイト生成（SSG）	Astro、Hugo、Next.js の静的出力
サーバー側テンプレート	Java + Thymeleaf、PHP（Laravel）、Ruby on Rails
クライアント側レンダリング（CSR）	React（Vite）、Vue、Next.js、Nuxt
サーバーとクライアントの併用（SSR）	Next.js、Nuxt

ほかにも：ISR（SSGで作ったページを、一定時間ごとやデータの更新時に作り直す。Next.js など）、CMS（WordPress など）、サーバー側テンプレートに部分的な通信を足す方式（htmx など）がある

方式ごとの作られ方

違いは「HTMLを、いつ・何が組み立てるか」

方式	HTMLの作られ方
静的Webページ	事前に、人が、HTMLを直接記述する
静的サイト生成（SSG）	事前に、人が、HTMLをコマンド等によって構築する
サーバー側テンプレート	リクエスト都度、サーバーが、テンプレートエンジンによって構築する
クライアント側レンダリング（CSR）	操作都度、ブラウザが、JavaScriptによって構築する
サーバーとクライアントの併用（SSR）	リクエスト都度、サーバーが構築するが、部分的に操作都度、ブラウザが、JavaScriptによっても構築する

静的（上の2つ）：配信する時点で、完成したHTMLがすでにある。**動的**（下の3つ）：アクセスのたびに、そのときのデータでHTMLを組み立てる

要求から選ぶ：何を優先したいか

要求	静的	SSG	テンプレート	CSR	SSR
見る人ごとに内容が変わる（ログイン後の画面）	×	×	○	○	○
再読み込みせずに細かく反応する（アプリのような操作）	△	△	△	○	○
最初の表示が速い・検索エンジンに内容が伝わる	○	○	○	△	○
サーバーの運用が軽い（ファイルの配信だけで済む）	○	○	△	○	△
仕組みが単純で、学ぶことが少ない	○	△	○	△	×

各技術の選択基準（参考）

公開範囲	システム	特徴	選択技術
インターネット全体	会社案内・ドキュメント	更新が少ない公開ページ	静的・SSG
インターネット全体	商品ページ・記事	検索から人を集めたい公開ページ	SSR・SSG
会員限定	会員向けのマイページ	ログイン後に使う。クローリング不要	CSR
社内限定	業務システム・管理画面	操作が多い。検索エンジンに載せない	CSR
社内限定	小規模な社内ツール・既存システムの改修	画面が少なく、凝った操作がない	テンプレート

要求を同じように満たせるなら、チームの状況で選ぶ

確認
フロントエンドの技術（JavaScript・TypeScript）を学び、使い続ける意欲と体制があるか

はい →

確認
Node.js のサーバーを運用し続けられるか（人手・監視・更新）

はい →

SSR
画面向けの中継（BFF）までTypeScriptで書ける

▼ いいえ

サーバー側テンプレート
消極的な選択。慣れたサーバー側の言語の範囲で画面を作る

▼ いいえ

CSR + API
画面はファイルとして配信（CDNなど）。運用するサーバーはAPIだけ

ほかの判断材料：既存システムや社内の標準、保守する期間と人の入れ替わり、採用しやすい技術か、ライブラリのサポートが続くか

正解は1つではない。**要求とチームに合うかで選び、選んだ理由を説明できる**ことが大切

研修環境

サンプルコードはリポジトリから取得し、Dockerコンテナで動かす

PCに用意するもの

Git・Docker（Docker Desktop など）・ブラウザ（Chrome）・エディタ（使い慣れたもの）

JDK・Maven・Node.js はPCにインストールしなくてよい。コンテナの中のものを使う

IDE（IntelliJ IDEA など）での編集・動作確認は任意

コンテナは、**それを使う部の冒頭で、その都度起動する**

研修環境

リポジトリの構成：部ごとのディレクトリ

ディレクトリ	内容	演習
html-js	第1部：タスク一覧のHTML/CSSと、Consoleで試すJavaScript	演習2〜4
template-engine	第2部：Spring Boot＋Thymeleaf のタスク一覧画面（task-web）	演習5・6
node	第3部：Express のタスクAPI（task-api）と、TypeScriptのサンプル	演習7〜10
react-next	第4部：React（task-ui）と Next.js（task-next）のタスク画面	演習11〜13

ファイル	役割
README.md	各部の起動方法。各ディレクトリの README.md には、ファイルとスライドの対応を書いている
template-engine/task-web/mvnw・mvnw.cmd・.mvn/	Maven Wrapper。./mvnw で Maven を実行する

スライドの下には、そのコードがあるファイルを示しています。演習も、このファイルに手を加えて取り組みます。

第1部

HTML/CSS復習 JavaScript

ブラウザが受け取るHTML・CSSを復習し、JavaScriptで画面に動きを付ける

ブラウザが理解できる言語は3つだけ

HTML

構造と意味

見出し、一覧、入力欄。
建物でいえば骨組み

CSS

見た目

色、大きさ、配置。
建物でいえば内装・外装

JavaScript

動き

クリックへの反応、通信、画面の書き換え。
建物でいえば電気・設備

Javaのサーバーが返すページのレスポンスも、TypeScript や React で作った画面も、最終的にはこの3つの形でブラウザに届く

演習1 ・ 開発者ツール（5分）

開発者ツール（F12）の3つのタブを開く

好きなWebサイトで F12 を押し、次の3つのタブを順に開いてみてください。この3つは研修を通してずっと使います。

- Elements：いま表示されているHTMLとCSS。その場で書き換えて試せる
- Console：JavaScriptを1行ずつ実行できる。エラーもここに出る
- Network：ブラウザとサーバーの通信。リクエストとレスポンスの中身が見える

開発者ツールはほとんどのブラウザにありますが、タブの表記はブラウザによって多少違います。上は Chrome の表記で、Firefox では Inspector・Console・Network です。

確認環境を用意する：nginxコンテナを起動する

nginx はWebサーバー。指定したフォルダの中のファイルを、そのままブラウザに配信する。リポジトリの html-js/html フォルダを配信させる

```
cd frontend-training   # リポジトリのフォルダで実行する
docker container run -d --name edu_html \
  -p 8000:80 \
  -v "$PWD/html-js/html:/usr/share/nginx/html:ro" \
  nginx:stable

docker container ls   # NAMES が edu_html の行の STATUS が Up なら成功
```

-v で、PCの html-js/html フォルダを、コンテナの中に共有している。PCのエディタで保存した内容が、そのままコンテナから配信される

--name で、コンテナに edu_html という名前を付けている。停止・再起動などの操作は、この名前で指定する

HTMLの例は、エディタで保存して再読み込みするだけで確かめられる

1. エディタで開く
html-js/html/tasks.html を開く

→

2. ブラウザで開く
`localhost:8000/tasks.html`

→

3. 直して F5
エディタで書き換えて保存し、ブラウザを再読み込み

接続確認用：html-js/html/hello.html

```
<!DOCTYPE html>
<html lang="ja">
<head><meta charset="UTF-8"></head>
<body><h1>こんにちは</h1></body>
</html>
```

nginxコンテナは html-js/html フォルダをそのまま配信するので、コンテナに入る必要はない。localhost:8000/ だけだと 403 になる（index.html が無いため）ので、ファイル名まで入力する

HTML：タグで囲んで意味を付ける

```html
<h1>タスク一覧</h1>
<p id="message"></p>

<ul class="task-list">
  <li class="done" data-id="1"><span>設計書を読む</span><button type="button">完了切替</button></li>
  <li data-id="2"><span>APIを作る</span><button type="button">完了切替</button></li>
</ul>

<form id="add-form">
  <input type="text" name="title" placeholder="新しいタスク">
  <button type="submit">追加</button>
</form>
```

id：ページ内で1つだけ
class：複数の要素に付けられる

data-*：要素に独自のデータを持たせる。後でJSから読む

name：フォーム送信時のパラメータ名になる

CSS：セレクタで選んで見た目を指定する

```
.task-list { list-style: none; padding: 0; }
.task-list li {
  display: flex; /* 子要素を横に並べる */
  justify-content: space-between;
  padding: 8px;
  border-bottom: 1px solid #ccc;
}
.task-list li.done span {
  text-decoration: line-through;
  color: #888;
}
#message { color: crimson; }
```

ボックスモデル：内容 → padding → border → margin の順に外側へ広がる箱。配置の調整はまず display: flex を試す。

CSSセレクタ早見表：スタイルを当てる要素を特定する

種類	書き方	例	選ばれるもの
要素	要素名	p	p の場合、すべての p 要素
クラス	.クラス名	.done	.done の場合、class に done を持つ要素（クラス名は自由）
ID	#ID名	#message	#message の場合、id が message の要素（ページ内に1つ）
属性	[属性名="値"]	[type="submit"]	[type="submit"] の場合、type 属性が submit の要素
疑似クラス	:状態	button:hover	button:hover の場合、マウスが乗っている状態の button
疑似要素	::位置	li::before	li::before の場合、li の中身の前の「場所」（要素ではない）

競合したら詳細度（強さ）で決まる **ID** > **クラス・属性・疑似クラス** > **要素・疑似要素**

<li class="done" data-id="1"> に3つのルールが当たると

```
/* 要素：弱い */
li { color: gray; }
/* クラス */
.done { color: green; }
/* 属性：クラスと同じ強さ */
[data-id="1"] { color: blue; }
```

結果：文字は青になる

【1】詳細度が**高い**方が勝つ：li は弱いので負ける
【2】詳細度が**同じ**なら**後に書いた**方が勝つ：.done と [data-id="1"] は同じ強さなので、後に書いた青が勝つ
.done の行を最後に移すと、緑に変わる

組み合わせると詳細度が上がる：3つの数で計算する

詳細度は **（ID の数, クラス・属性・疑似クラスの数, 要素・疑似要素の数）** の3つの数で表す。組み合わせたセレクタは、使った分だけ足し算する。

セレクタ	選ばれるもの	詳細度
div p	div の中（子孫）にある p	(0, 0, 2)
table > thead	table の直下（子）にある thead	(0, 0, 2)
.message	class に message を持つ要素	(0, 1, 0)
div.message	class に message を持つ div（div かつ .message）	(0, 1, 1)
ul.task-list li.done	task-list の ul の中にある、done を持つ li	(0, 2, 2)
#add-form button:hover	#add-form の中で、マウスが乗っている button	(1, 1, 1)

```
.message { color: gray; }          /* (0, 1, 0) */
div.message { color: crimson; }    /* (0, 1, 1) ← 要素が1つ多いので、こちらが勝って赤になる */
```

左の数から比べ、大きい方が勝つ。数は繰り上がらないので、クラス11個の (0, 11, 0) も ID 1つの (1, 0, 0) に負ける

空白・> などの階層の記号は数えない。**div.message**（div かつ .message）と **div .message**（div の中の .message）は、空白1つで意味が変わる

（補足）位置で選ぶ：first- / last- / nth- と -child / -of-type

-child は兄弟の要素すべてを数えて何番目かを見る。**-of-type** は同じ種類の要素だけを数える。どちらも疑似クラスなので、詳細度はクラスと同じ。

セレクタ	選ばれるもの
li:first-child	最初の子要素が li なら、その li（last-child は最後）
li:nth-child(2)	2番目の子要素が li なら、その li
tr:nth-child(odd)	奇数番目の tr（even は偶数番目）。表の縞模様などに使う
p:first-of-type	兄弟の p の中で最初の p（last-of-type は最後）
p:nth-of-type(2)	兄弟の p の中で2番目の p

```
<div class="card">
  <h3>見出し</h3>
  <p>1つ目の段落</p>   <!-- p:first-of-type、p:nth-child(2) に当たる -->
  <p>2つ目の段落</p>   <!-- p:last-child、p:last-of-type に当たる -->
</div>
```

この例の **p:first-child** は何も選ばない。最初の子要素は h3 で、p ではないため

（補足）!important：詳細度よりも優先される

値の後ろに **!important** を付けた宣言は、詳細度に関係なく、付けていない宣言よりも優先される。

<p id="message"> に両方が当たると：詳細度が低くても、!important が勝つ

```
#message { color: blue; }        /* (1, 0, 0) */
p { color: gray !important; }    /* (0, 0, 1) だが !important なので、こちらが勝つ */
```

<p class="note"> に両方が当たると：!important 同士なら、詳細度が高い方が勝つ

```
p.note { color: red !important; }   /* (0, 1, 1) ← 詳細度が高いので、こちらが勝つ */
p { color: gray !important; }       /* (0, 0, 1) */
```

詳細度も同じなら、通常と同じく後に書いた方が勝つ。

注意：極力使わない

!important を付けると、詳細度を上げただけでは上書きできなくなる。上書きするには、さらに !important が必要になり、連鎖していく

スタイルの調整は、セレクタの詳細度と書く順番で行う

演習2 ・ HTML/CSS（10分）

tasks.html を、手で書き換える

次の手順で、HTMLとCSSを手で書き換えてみてください。

1. エディタで html-js/html/tasks.html を開き、ブラウザで localhost:8000/tasks.html を開く
2. Elements タブで li を選び、class="done" を手で付け外しする
3. 3件目のタスクを追加する（HTMLを書き足して保存し、F5 で再読み込みする）
4. style の中に button:hover { … } を追加し、マウスを乗せたとき背景色を変える

ファイル：html-js/html/tasks.html

手順2がいちばん重要

class を付け外しすると見た目が変わる。JavaScriptで画面を動かす基本は、これをプログラムから行うこと

JavaScriptはJavaとは別の言語

観点	Java	JavaScript
型が決まるとき	コンパイル時（変数に型がある）	実行時（値に型がある。変数には無い）
実行方法	コンパイルしてJVMで実行	ブラウザやNode.jsがそのまま実行
コードの置き場所	必ずクラスの中	ファイルに直接書ける
関数	メソッド（クラスに所属）	関数そのものが値。変数に入れられる
オブジェクトの作成	クラスを定義して new	{} でその場で作れる
数値	int、long、double など	number の1種類（double 相当）
値の比較	==	===（== は型の一致まで確認しないため、"1" == 1 も true になる）
オブジェクトの比較	equals()（== は同じインスタンスかどうか）	=== は同じオブジェクトかどうか。文字列は値なので === で中身を比べられる

if・for・while・switch・try/catch はほぼ同じ。ここでは違う部分だけを扱います。試す場所はConsoleタブ。

動く場所は2つ：言語は同じ、使える機能が違う

ブラウザ

画面を動かす。第1部・第2部・第4部（画面側）

DOM API：画面のHTML要素を読み書きする
document.querySelector など

使えない：PCのファイルやOS

Node.js

サーバーや開発ツールを動かす。第3部・第4部

fs・http・process：ファイル、サーバー、環境変数

使えない：document・window（画面が無い）

どちらでも同じ：言語そのもの、console.log、setTimeout、fetch。
このあと学ぶ変数・配列・Falsyなどは言語の機能だけなので、ConsoleでもNode.jsでも同じように動く

変数は const と let。文字列には複数の書き方がある

```
const title = "設計書を読む"; // 再代入できない（Javaの final に近い）
let count = 0; // 再代入できる
count = count + 1;
count = "いっぱい"; // エラーにならない。変数に型がないため

const a = "ダブルクォート"; // " で囲む
const b = 'シングルクォート'; // ' で囲む。" と同じ意味で、どちらもよく使われる
const name = "佐藤";
const c = `${name}さん、こんにちは`; // ` で囲むと ${ } で値を埋め込める（テンプレートリテラル）

const d = /^task/; // / で囲むと正規表現（RegExp オブジェクト）
```

基本は **const**。再代入が必要なときだけ **let**。古いコードの **var** は、新しく書くコードでは使わない

/ / で囲む書き方は、文字列として機能する部分もありますが、実際には正規表現を表します。テンプレートリテラルが無かった時期には文字列の代用として使われることもありましたが、新しく書くコードで文字列に使うことはありません

オブジェクトはクラスなしで作れる

```
const task = { id: 1, title: "設計書を読む", done: false };

console.log(task.title); // 設計書を読む
task.done = true; // const でも中身は変更できる（Javaの final な参照と同じ）
task.memo = "急ぎ"; // 存在しないプロパティは追加される
console.log(task.owner); // undefined（エラーにはならない）
```

この { } の書き方が、**JSON** の元になった

存在しないプロパティを読んでも例外にならず **undefined** が返る。このあと学ぶFalsyや ?. の前提になる

関数は「値」として扱える

```
// 関数宣言：引数にも戻り値にも型を書かない
function label(task) {
  return `#${task.id} ${task.title}`;
}

// アロー関数：Javaのラムダ式に相当（矢印は -> ではなく =>）
const isOpen = (task) => !task.done;

const task = { id: 2, title: "APIを作る", done: false };
console.log(label(task)); // #2 APIを作る
console.log(isOpen(task)); // true
```

関数を変数に入れられる ＝「処理そのものを引数として渡す」書き方ができる。
配列のメソッド、イベント処理、Reactで何度も登場する

配列：ループよりメソッドを使う

```
const tasks = [
  { id: 1, title: "設計書を読む", done: true },
  { id: 2, title: "APIを作る", done: false },
  { id: 3, title: "画面を作る", done: false },
];

tasks.filter((t) => !t.done); // 未完了の2件だけの配列
tasks.map((t) => t.title); // ["設計書を読む", "APIを作る", "画面を作る"]
tasks.find((t) => t.id === 2); // idが2のタスク（無ければ undefined）
tasks.some((t) => t.done); // true（1件でも完了があるか）

// つなげて書ける
tasks.filter((t) => !t.done).map((t) => t.title); // ["APIを作る", "画面を作る"]

// 順番に処理するだけなら for...of（Javaの拡張for文に相当）
for (const t of tasks) { console.log(t.title); }
```

JavaのStream APIにあたる操作が配列に直接ある。stream() も collect() も不要。

分割代入とスプレッド構文

```
const task = { id: 1, title: "設計書を読む", done: true };
const tasks = [task];

// 分割代入：プロパティを変数に取り出す
const { id, title } = task;

// オブジェクトのスプレッド：コピーして一部だけ変える（元の task は変更されない）
// 後ろに書いた同名のプロパティ（done）は、値の「置き換え」になる
const reopened = { ...task, done: false };
// → { id: 1, title: "設計書を読む", done: false }

// 配列のスプレッド：後ろに書いた要素は「追加」になる（置き換えではない）
const more = [...tasks, { id: 4, title: "テストする", done: false }];
// → 要素が2件の新しい配列。元の tasks は1件のまま
```

第4部のReactで毎回使う。「元を書き換えず、新しいものを作る」書き方に慣れておく

演習3 ・ 配列メソッド（15分）

配列メソッドで tasks を加工する

まず html-js/js/ex03-array.js の tasks を Console に貼り付ける

```
const tasks = [
  { id: 1, title: "設計書を読む", done: true },
  { id: 2, title: "APIを作る", done: false },
  { id: 3, title: "画面を作る", done: false },
];
```

続けて、次の3つをファイルのコメントの下に書き、1つずつ Console で実行してください。

1. 完了済みタスクの件数を求める
2. 「#1 設計書を読む（完了）」「#2 APIを作る（未完了）」の形式の文字列の配列を作る
3. idが3のタスクだけを完了にした、新しい配列を作る（元の tasks は変更しない）

型は「値」に付いている

```
typeof 42;          // "number"
typeof "abc";       // "string"
typeof true;        // "boolean"
typeof undefined;   // "undefined"
typeof { id: 1 };   // "object"
typeof [1, 2];      // "object" ← 配列も object。配列かどうかは Array.isArray(x) で判定する
typeof (() => 1);   // "function"
```

型は8種類。よく使うのは number・string・boolean・undefined・null・object の6つ

「無い」が2種類ある。通常の実装で使うのは undefined

undefined

値が無い。通常の実装時はこちら

代入前の変数、存在しないプロパティ、渡されなかった引数、return の無い関数、find で見つからないとき

null

外から来る「無い」

APIのJSON（JSONに undefined は無い）、ブラウザの DOM API（querySelector で見つからないとき）。通常の実装では作らない

React・Next.js・TypeScriptでは「無い」を undefined で表すのが一般的。
受け取った null も、?. や ?? を使えば undefined と同じように扱える

Falsy：if で false 扱いになる6つの値

false	0	""	null	undefined	NaN
偽	ゼロ	空文字列	外から来る無	まだ無い	数値変換の失敗

おおむね「空っぽ・無い・ゼロ」を表す値。これ以外はすべて Truthy（true 扱い）。

```
Boolean("0");       // true ← 空でない文字列はすべてTruthy
Boolean("false");   // true
Boolean([]);        // true ← 空の配列でもTruthy。空かどうかは tasks.length === 0 で判定する
Boolean({});        // true ← 空のオブジェクトでもTruthy
```

注意：上の4つは、直感に反して Truthy になる

入力チェックを1行で書く

```
function validate(title) {
  if (!title?.trim()) {   // Javaなら title != null && !title.isBlank()
    return "タイトルは必須です";
  }
  return undefined;   // 問題なし
}

validate(undefined);    // "タイトルは必須です"
validate("");           // "タイトルは必須です"
validate("   ");        // "タイトルは必須です"
validate("APIを作る");  // undefined
```

?. は、左が null か undefined ならそこで止まり、undefined を返す

undefined も "" も Falsy なので、**!** ひとつで3つのケースを弾ける

デフォルト値を || で与える

```
function greet(user) {
  const name = user?.profile?.nickname || user?.name || "ゲスト";
  return `こんにちは、${name}さん`;
}

greet(); // こんにちは、ゲストさん
greet({ name: "佐藤" }); // こんにちは、佐藤さん
greet({ name: "佐藤", profile: { nickname: "" } }); // こんにちは、佐藤さん（空文字は飛ばす）
greet({ name: "佐藤", profile: { nickname: "さとう" } }); // こんにちは、さとうさん
```

a || b は「a がFalsyなら b」。
boolean ではなく、値そのものを返す

「無いものはすべてFalsy」だから、優先順位を左から並べるだけで済む。Javaでは if が何段も重なる

0 や "" が正しい値のときは ?? を使う

```
const query = { size: 0 };
query.size || 20;   // 20 ← 0を指定したのに
query.size ?? 20;   // 0

const settings = { debug: false };
settings.debug || true;   // true ← 誤り
settings.debug ?? true;   // false
```

?? は、左が null か undefined のときだけ右を返す。0・""・false はそのまま通す

やりたいこと	書き方
空・ゼロも含めて「無ければ」代わりの値（表示名など）	a || b
未指定のときだけデフォルト値（件数、フラグ、金額）	a ?? b
無いかもしれないもののプロパティを安全に読む	a?.b
値があるときだけ処理する	if (a) { }

比較は常に ===、型の違いは計算の前に自分で変換

```
"1" == 1;   // true ← 型の一致まで確認しない
0 == "";    // true
"1" === 1;  // false ← こちらを使う

"5" + 2;          // "52" ← 連結
"5" * 2;          // 10
Number("5") + 2;  // 7
Number("abc");    // NaN（例外ではない）
Number("");       // 0 ← 注意
```

常に **===** と **!==** を使う、と決めてしまえば迷わない

画面の入力欄から取れる値は、数字であっても**必ず文字列**

演習4 ・ Falsy（15分）

予想してから、Consoleで確かめる

ファイル：html-js/js/ex04-falsy.js（1行ずつ Console に貼り付けて実行する）

問1　結果を紙に書いてから、Console で実行する

```
Boolean(" "); // 【1】 空白1文字
Boolean("0"); // 【2】
Boolean(Number("0")); // 【3】
["a", "", undefined, "b"].filter(Boolean); // 【4】
undefined ?? "x"; // 【5】
"" ?? "x"; // 【6】
"" || "x"; // 【7】
```

問2　関数の中身を1行で書く

```
// task.assignee は { name: "佐藤" } のことも、プロパティ自体が無い（undefined の）こともある
// 担当者名を返す。取れなければ "未割当" を返す
function assigneeName(task) {
  // ここを1行で書く
}
```

画面を動かすために必要な手続き

1. 要素を特定する
CSSセレクタで要素を取得する

→

2. 操作を待つ
クリックされたら呼ぶ関数を預ける

→

3. 要素を書き換える
文字やclassを変える、要素を足す

ステップ	書き方	説明
特定する	document.querySelector("#message")	最初の1件。無ければ null
操作を待つ	element.addEventListener("click", 関数)	クリックされたら関数を呼ぶ
書き換える	element.textContent = "..."	中の文字を変える
書き換える	element.classList.toggle("done")	classを付け外しする
作る	document.createElement("li")	append() で画面に追加

tasks.html にJavaScriptを足す

```
const list = document.querySelector(".task-list");
const form = document.querySelector("#add-form");
const message = document.querySelector("#message");

// 完了切替：親の ul にまとめて登録する
list.addEventListener("click", (event) => {
  if (!event.target.matches("button")) return;
  event.target.closest("li").classList.toggle("done");   // 演習2で手作業で行った class の付け外し
});

form.addEventListener("submit", (event) => {
  event.preventDefault();   // ページ再読み込みを止める
  const title = form.elements.title.value.trim();
  message.textContent = title ? "" : "タイトルは必須です";
  if (!title) return;   // 空文字列はFalsy
  // createElement で li を作り、list.append(li) する
});
```

安全上の注意：ユーザー入力を画面に出すときは textContent を使う。innerHTML はXSSの原因になる

fetch：ページを再読み込みせずサーバーを呼ぶ

練習用の公開サンプルAPI（JSONPlaceholder）を呼ぶ。Console に貼り付けてそのまま試せる

```
async function loadTodos() {
  const res = await fetch("https://jsonplaceholder.typicode.com/todos?_limit=3"); // 戻り値型は Promise<Response>
  if (!res.ok) throw new Error(`取得に失敗しました: ${res.status}`); // 200番台以外
  const todos = await res.json(); // JSONを変換。戻り値型は Promise<any>
  console.log(todos); // [{ userId: 1, id: 1, title: '…', completed: false }, …]
}
await loadTodos();
// 送信する場合（練習用のAPIなので、実際には保存されない）
const created = await fetch("https://jsonplaceholder.typicode.com/todos", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "画面を作る", completed: false }),
});
console.log(created.status, await created.json()); // 201 と、送った内容に id が付いたもの
```

戻り値型が **Promise** の関数を呼ぶときは **await** を付ける（関数には **async**）。詳しくは第3部

第2部

テンプレートエンジン

Javaのサーバーが組み立てたHTMLに、JavaScriptで動きを付ける

HTMLのひな形に、Javaのデータを流し込む

担当	やること	動く場所
Java・Spring MVC（Controller）	データを用意してテンプレートに渡す	サーバー
Java・Thymeleaf	テンプレートとデータからHTMLを組み立てる	サーバー
JavaScript	クリックへの反応、部分的な更新	ブラウザ

Javaのプロジェクトを、Corretto のコンテナで起動する

```
cd frontend-training
docker container run -d --name edu_java \
  -p 8080:8080 \
  -v "$PWD/template-engine/task-web:/work" -v ~/.m2:/root/.m2 -w /work \
  amazoncorretto:21 ./mvnw spring-boot:run

docker container logs -f edu_java   # 起動ログを見る（Ctrl+C でログの表示だけを終える）
```

オプション・引数	意味
--name edu_java	コンテナに付ける名前。ログの表示や再起動は、この名前で指定する
-w /work	コンテナ内で最初にいるフォルダ（PCの task-web フォルダと同じ中身）
-v ~/.m2:/root/.m2	PCの ~/.m2 を共有し、Maven 本体とライブラリをPCに保存する。コンテナを作り直しても再取得にならない
./mvnw spring-boot:run	イメージ名の後ろに書いたコマンドが、コンテナの中で実行される

http://localhost:8080/tasks を開く。コードを変更したら **docker container restart edu_java** で起動し直す

（補足）IDE（IntelliJ IDEA など）で編集してもよい。PCに Java 21 があれば、IDE から ./mvnw spring-boot:run で確かめてもよい

template-engine/task-web/mvnw
template-engine/task-web/pom.xml

Java側【1】：画面を返す処理

```java
// 画面を返す：戻り値はテンプレート名
@GetMapping("/tasks")
public String list(Model model) {
  model.addAttribute("tasks", tasks);
  return "tasks"; // templates/tasks.html が使われる
}

// フォーム送信を受け取る：処理後は一覧へリダイレクト
@PostMapping("/tasks")
public String add(@RequestParam("title") String title) {
  if (!title.isBlank()) {
    tasks.add(new Task(nextId++, title.trim(), false));
  }
  return "redirect:/tasks";
}
```

@Controller のメソッドは、戻り値の文字列を**テンプレート名**として扱う

"redirect:" で始めると、ブラウザに別のURLを開き直させる

Java側【2】：JSONを返す処理（1/2）

```java
// 一覧をJSONで返す
@GetMapping("/api/tasks")
@ResponseBody   // 戻り値をテンプレート名ではなくJSONとして返す
public List<Task> listApi() {
  return tasks;
}

// ボディのJSON {"title":"..."} を受け取る型
record NewTask(String title) {}

// JSONを受け取って追加する
@PostMapping("/api/tasks")
@ResponseBody
@ResponseStatus(HttpStatus.CREATED)   // 201 Created を返す
public Task create(@RequestBody NewTask body) {   // ボディのJSONを NewTask に変換する
  ...   // title が空なら 400
  tasks.add(task);
  return task;
}
```

Java側【2】：JSONを返す処理（2/2）

```
// 完了状態を切り替える
@PostMapping("/api/tasks/{id}/toggle")
@ResponseBody
public Task toggle(@PathVariable("id") int id) {
  ... task.setDone(!task.isDone());
  return task;   // 切り替えた後のタスク1件をJSONで返す
}
```

JavaScriptからは fetch("/api/tasks") で呼び出せる。第3部では、同じAPIをNode.jsで作る

テンプレート：普通のHTMLに th: 属性を足す

```
<p th:if="${tasks.isEmpty()}">タスクはありません</p>

<ul class="task-list">
  <li th:each="task : ${tasks}"
      th:classappend="${task.done} ? 'done'"
      th:data-id="${task.id}">
    <span th:text="${task.title}">サンプルのタスク</span>
    <button type="button">完了切替</button>
  </li>
</ul>

<form th:action="@{/tasks}" method="post">
  <input type="text" name="title" placeholder="新しいタスク" required>
  <button type="submit">追加</button>
</form>

<script th:src="@{/js/tasks.js}"></script>
```

th: 属性はJavaの構文に対応する

属性	働き	Javaでいうと
th:text="${task.title}"	要素の中身を値で置き換える（自動エスケープ）	getTitle() の出力
th:each="task : ${tasks}"	要素を件数分くり返す	拡張for文
th:if="..."	条件が真のときだけ出力する	if文
th:classappend="..."	条件に応じてclassを追加する	三項演算子
th:data-id="${task.id}"	data-id 属性に値を入れる（JSへの受け渡し）	
@{/tasks}	URLを組み立てる	

ブラウザに届くのは展開済みの普通のHTML。ブラウザはテンプレートエンジンの存在を知らない

JavaScriptを結合する（static/js/tasks.js）

```
const list = document.querySelector(".task-list");
const message = document.querySelector("#message");

list.addEventListener("click", async (event) => {
  if (!event.target.matches("button")) return;

  const li = event.target.closest("li");
  const res = await fetch(`/api/tasks/${li.dataset.id}/toggle`, { method: "POST" });
  if (!res.ok) {
    message.textContent = "更新に失敗しました";
    return;
  }
  const task = await res.json();
  li.classList.toggle("done", task.done); // サーバーの結果に合わせて付け外しする
});
```

サーバーが **data-id** に埋め込んだIDを、JavaScriptが **li.dataset.id** で読んでAPIを呼ぶ（値は文字列）

演習5 ・ テンプレートエンジン（25分）

「追加」と「完了切替」の通信を比べる

次の手順で、2つの操作の通信を比べてください。

1. 第2部の最初に起動した Java のコンテナ（edu_java）が動いている状態で、localhost:8080/tasks を開き、追加と完了切替が動くことを確認する
2. Network タブを開いたまま「追加」と「完了切替」を行い、通信を比べる
3. 完了切替の後にページを再読み込みし、状態が保持されていることを確認する

template-engine/task-web/src/main/resources/templates/tasks.html
template-engine/task-web/src/main/resources/static/js/tasks.js

演習6 ・ fetch（10分）

Console から API を直接呼ぶ（1/2）

1. localhost:8080/tasks を開き、F12 で開発者ツールを開いて Console タブを選ぶ
2. 【A】を貼り付けて Enter。タスク一覧のJSON（配列）が表示される
3. 【B】を貼り付けて Enter。201 と、追加されたタスクが表示される

【A】一覧を取得する（GET）

```
const res = await fetch("/api/tasks");
const tasks = await res.json();
console.log(tasks);
```

【B】タスクを追加する（POST）

```
const created = await fetch("/api/tasks", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "画面を作る" }),
});
console.log(created.status, await created.json());
```

演習6 ・ fetch（10分）

Console から API を直接呼ぶ（2/2）

4. ページを再読み込みし、一覧に「画面を作る」が増えたことを確かめる
5. 【C】を実行し、await を付けないと中身ではなく Promise が入ることを確かめる

【C】await を付けないと

```
const p = fetch("/api/tasks");
console.log(p); // Promise {<pending>}
```

うまく貼り付けられないとき

貼り付けが止められて警告が出たら、表示の指示どおり allow pasting と入力して Enter を押し、貼り付け直す

第1部・第2部 ・ まとめ

第1部・第2部でおさえた3つのこと

1. ブラウザが理解するのは HTML（構造）・CSS（見た目）・JavaScript（動き）の3つ

2. Falsyな6つの値と ||・??・?. を使うと、「無いとき」の処理を短く書ける

3. テンプレートエンジン方式ではサーバーが完成したHTMLを返す。JSは一部だけ動かしたいところに足す

次の部への問い：すべての操作をJSONのやり取りにしたら、サーバーは何を返せばよいか？

第3部

Node.js TypeScript

JavaScriptをブラウザの外で動かしてAPIサーバーを作り、それに型を付ける

Node.js環境を用意する：以降はこのコンテナで作業する

Node.js 24（LTS）と npm・curl・git が入ったコンテナを使う。リポジトリのフォルダ全体を共有し、第3部・第4部のプロジェクトをこの中で動かす

```
cd frontend-training
docker container run -d --name edu_frontend \
  -p 3000:3000 -p 3001:3001 -p 5173:5173 \
  -v "$PWD:/work" -w /work \
  node:24 sleep infinity
```

オプション・引数	意味
--name edu_frontend	コンテナに付ける名前。中に入るとき（exec）は、この名前で指定する
-p 3000:3000 …	3001 はAPI、5173 と 3000 は第4部の画面。使うポートをまとめて公開しておく
-v "$PWD:/work"	リポジトリのフォルダ全体を共有する
sleep infinity	何もせず待ち続けさせる。中には exec で入って作業する

以降の node・npm・npx・curl は、すべて edu_frontend コンテナの中で実行する

コンテナには exec で入ってコマンドを実行する

```
docker container exec -it edu_frontend bash
node -v   # v24 と表示される
cd node/task-api
npm install   # package.json に書かれたライブラリを取得する
exit   # 抜ける（コンテナは停止しない）
```

部分	意味
exec	起動中のコンテナの中でコマンドを実行する
-i	キーボード入力をコンテナに渡す
-t	端末として接続する（プロンプトが出る）
edu_frontend	接続先のコンテナ名
bash	中で実行するコマンド（シェル）

コンテナで作業するときの3つの注意

1

ターミナルは複数開く

サーバーを起動したターミナルはログ表示で占有される。別のターミナルを開き、同じ exec コマンドでもう1本接続する

2

届くのは公開ポートだけ

3001 はAPI、5173 と 3000 は 第4部の画面。公開したポート以外にはPCのブラウザから届かない

3

Vite は --host を付ける

Viteは既定でコンテナ内部からの接続しか受け付けない。第4部では npm run dev -- --host で起動する

```
docker container stop edu_frontend  # 作業を終えるときに停止
docker container start edu_frontend # 作業を再開するときに起動
```

Node.js ＝ ブラウザの外でJavaScriptを動かす実行環境

学ぶ理由 1

フロントエンド開発の道具がすべてNode.jsで動く

TypeScriptの変換、Reactのビルド、開発用サーバー。第4部はこれが無いと始められない

学ぶ理由 2

サーバーそのものも書ける

画面と同じ言語でAPIを書ける。小規模なAPIや、画面向けの中継サーバーによく使われる

観点	ブラウザのJavaScript	Node.js
できること	画面（DOM）の操作	ファイル読み書き、サーバー起動、DB接続
使えないもの	ファイルやOSへの自由なアクセス	document や window（画面が無い）
共通	言語そのもの、console.log、fetch	（同じ）

演習7 ・ Node.js（10分）

コンテナの中で、最初のJavaScriptを実行する

1. 3-1で起動した edu_frontend コンテナに接続して、node/task-api フォルダへ移動する

```
docker container exec -it edu_frontend bash
cd node/task-api
```

2. hello.js をエディタで開いて読む

```
const tasks = [
  { id: 1, title: "設計書を読む", done: true },
  { id: 2, title: "APIを作る", done: false },
];
const open = tasks.filter((t) => !t.done).map((t) => t.title);
console.log(`未完了: ${open.join(", ")}`);
```

3. 実行する。続けて、tasks に未完了のタスクを1件追加して保存し、もう一度実行する

```
node hello.js
# 未完了: APIを作る
```

node/task-api/hello.js

npm は Maven に対応づけて理解する

役割	Maven	npm
プロジェクト定義ファイル	pom.xml	package.json
依存ライブラリの追加	<dependency> を記述	npm install パッケージ名
ライブラリの保存先	~/.m2（PC全体で共有）	node_modules（プロジェクトごと）
バージョンの固定	pom.xml に明記	package-lock.json（自動生成）
コマンドの実行	mvn package など	npm run スクリプト名
公開リポジトリ	Maven Central	npm registry

node_modules はGitに含めない。package.json と package-lock.json があれば、npm install で復元できる

プロジェクトの作り方と、ファイルの分け方

```
# このプロジェクトは、次のコマンドで作成した（実行済み）
npm init -y # package.json を作成
npm pkg set type=module # import / export を使う設定（"type": "module"）
npm install express cors # ライブラリを追加（このあとAPIサーバーで使う）
```

taskUtil.js：export を付けたものだけ外から使える

```
export function openTitles(tasks) {
  return tasks
    .filter((t) => !t.done)
    .map((t) => t.title);
}
```

hello-module.js：import して使う

```
// 自分のファイルは ./ と拡張子を付ける
import { openTitles } from "./taskUtil.js";
// ライブラリは名前だけ
import express from "express";
```

Java と比べると：export を付けたものは Java の public のように、ほかのファイルから使える。付けないものは private のように、そのファイルの中だけで使える。公開の単位が、Java ではクラス、JavaScript ではファイルになる

やってみよう（10分） hello.js を、taskUtil.js の openTitles を import して使う形に書き換える（完成形は hello-module.js）

node/task-api/package.json
node/task-api/taskUtil.js
node/task-api/hello-module.js

待ち時間の間に、次のリクエストを進める

観点	Java（Spring MVCの標準構成）	Node.js
DBや通信の待ち時間	そのリクエストの処理が止まって待つ	待たずに次のリクエストへ進み、結果が届いたら続きを実行
同時に進められるリクエストの数	あらかじめ決めた上限の件数まで	待ち時間の間に次へ進むので、上限が大きい（次のスライド）
DBや通信を呼び出すコードの書き方	普通のメソッド呼び出しとして書く。結果が返るまで次の行に進まない	戻り値型が Promise の関数を呼ぶときに await を付ける。await の行で待つ間、ほかのリクエストが進む
注意点	複数のリクエストが同じデータを同時に書き換える競合	重い計算をすると、そのプロセスの全リクエストが止まる

「あとで結果が届く」ことを表す値が **Promise**。JavaのCompletableFuture にあたる

戻り値型が Promise の関数を呼ぶときは **await** を付ける。
結果が届くまで関数の続きを保留し、中身を取り出せる

同時に進められるリクエストの数で比べる

例：1件のリクエストが「計算 1ミリ秒 ＋ 外部APIの応答待ち 500ミリ秒」かかる場合

計算＝CPUが働く部分。リクエストのJSONを読む、入力をチェックする、DBの結果をJSONにして返す、など。一般的なAPIでは、数十件のデータを扱う程度なら1ミリ秒前後で終わることが多い（目安）

観点	Java（Spring MVCの標準構成）	Node.js
同時に進められるリクエストの上限	200件（Tomcat の標準設定）	決まった上限はない。待っているリクエストはメモリをわずかに使うだけ
1秒あたりに処理できる件数	約400件（200件が0.5秒ずつ上限を占める）	約1000件（計算が1件1ミリ秒のため）
待ち時間が1秒になると	約200件に減る	約1000件のまま（同時に進める件数が増えるだけ）
上限を超えたリクエスト	空きが出るまで待たされる	計算が追いつかない分だけ遅くなる

APIサーバーは、ほとんどの時間を DB や外部APIの応答待ちに使う。待ちが長いほど、Java との差が開く

計算が長い処理では逆になる。例：大量データの集計（4項目×10万件で数十ミリ秒、×100万件で数百ミリ秒）、大きなJSONの変換（20項目×1万件・約4MBで十数ミリ秒、×10万件・約40MBで百ミリ秒以上）。複数の CPU で同時に計算できる Java のほうが多く処理できる

演習8 ・ 非同期処理（5分）

出力される順番を予想する

```
// async1.js
console.log("A");

// 1秒後に関数を実行するよう予約
setTimeout(() => console.log("B"), 1000);

console.log("C");
```

Javaの感覚だと

A → B → C ？

戻り値型が Promise なら await。付け忘れに注意

```
// async2.js
import { readFile } from "node:fs/promises";

const text = await readFile("package.json", "utf8");   // 戻り値型は Promise<string>
const pkg = JSON.parse(text);
console.log(pkg.name);   // task-api

// 失敗は try/catch で受ける（Javaと同じ形）
try {
  await readFile("存在しない.txt", "utf8");
} catch (e) {
  console.log(`読めませんでした: ${e.code}`);   // 読めませんでした: ENOENT
}
```

```
const text = readFile("package.json", "utf8");   // await が無い
console.log(text);   // Promise { <pending> }
```

最もよく踏むミス：中身ではなく Promise そのものが変数に入る

await を付けない使い方：いつ待つかは呼び出す側が決める

1. 複数の処理を先に始めて、あとでまとめて待つ（Promise.all は次のスライドで詳しく）

```
const pkgPromise = loadText("package.json");   // 待たずに読み込みを始める
const helloPromise = loadText("hello.js");   // 1つ目を待たずに、2つ目も始まる
const [pkgText, helloText] = await Promise.all([pkgPromise, helloPromise]);   // ここでまとめて待つ
```

2. 結果を待つ必要がない処理（ログの書き込みなど）

```
appendFile("access.log", "async4.js を実行しました\n");   // 書き込みの完了を待たずに次へ進む
console.log("ログの書き込みを待たずに、ここまで進みました");
```

3. Promise のまま呼び出し元に返す

```
function loadText(path) {
  return readFile(path, "utf8");   // 戻り値型は Promise<string>。待つのは呼び出し元
}
```

Promise を返すのは、呼び出した側が「待つか・いつ待つか」を選べるようにするため。結果の値を使うときは、最後に必ず await（または Promise.all）で待つ

複数の処理を同時に待つ：Promise.all

```
import { setTimeout as sleep } from "node:timers/promises";   // 指定したミリ秒後に完了する Promise を返す（Node.js 標準）

console.time("順番に");
const a = await sleep(1000, "A");   // 1秒かかる通信の代わり。1秒後に "A" が届く
const b = await sleep(1000, "B");
console.timeEnd("順番に");   // 約2秒

console.time("同時に");
// Promise.all の戻り値型も Promise なので await する
const [c, d] = await Promise.all([sleep(1000, "C"), sleep(1000, "D")]);
console.timeEnd("同時に");   // 約1秒
```

Express は Spring Boot のコントローラーと見比べる

Spring Boot	Express
@GetMapping("/api/tasks")	app.get("/api/tasks", (req, res) => { ... })
@PathVariable("id")	req.params.id（常に文字列）
@RequestBody	req.body
戻り値がJSONになる	res.json(値)
ResponseEntity.status(404)	res.status(404)

ここで作るAPI（ポート3001）

```
GET /api/tasks
GET /api/tasks/:id
POST /api/tasks
PATCH /api/tasks/:id
```

「すべての操作をJSONのやり取りにしたら、サーバーは何を返せばよいか」への答えが、HTMLを返さず**JSONだけ**を返す、このAPIサーバー

最近はフロントエンドとバックエンドの分業が進み、Spring MVC で作るバックエンドも、JSONだけを返すAPI専用のサーバーとして作ることが少なくない

server.js（1/2）：準備と一覧取得

```
import express from "express";
import cors from "cors";

const app = express();
app.use(cors()); // 別オリジン（第4部の画面）からの呼び出しを許可する
app.use(express.json()); // リクエストボディのJSONを req.body に入れる

let tasks = [   // DBの代わり。削除で入れ直すので let にする
  { id: 1, title: "設計書を読む", done: true },
  { id: 2, title: "APIを作る", done: false },
];
let nextId = 3;

app.get("/api/tasks", (req, res) => {
  res.json(tasks);
});
app.listen(3001, () => console.log("http://localhost:3001 で起動しました"));
```

```
node --watch server.js   # ファイル保存のたびに自動で再起動する
```

server.js（2/2）：値の取り出し・変換・検証は自分で書く

```
// 追加
app.post("/api/tasks", (req, res) => {
  const title = req.body?.title?.trim();
  if (!title) {   // ボディ無し・title無し・空文字・空白だけ、の4ケースをまとめて400にする
    return res.status(400).json({ message: "title は必須です" });
  }
  const task = { id: nextId++, title, done: false };
  tasks.push(task);
  res.status(201).json(task);
});

// 完了状態の更新
app.patch("/api/tasks/:id", (req, res) => {
  const task = tasks.find((t) => t.id === Number(req.params.id));   // req.params.id は文字列なので数値に変換
  if (!task) {   // find は見つからないと undefined を返す
    return res.status(404).json({ message: "タスクが見つかりません" });
  }
  task.done = req.body?.done ?? !task.done;   // false の指定を正しく扱うため、|| ではなく ??
  res.json(task);
});
```

Spring MVC では、@RequestBody・@PathVariable の型への変換や @Valid による検証をフレームワークが行う。Express は受け取った値をそのまま渡すので、取り出し・変換・検証をコードで書く

node/task-api/server.js

演習9 ・ Express（30分）

APIを呼び出し、削除を追加する

1. ターミナルをもう1本コンテナに接続し、APIを呼び出す

```
docker container exec -it edu_frontend bash # 2本目のターミナルで

curl http://localhost:3001/api/tasks
curl -X POST http://localhost:3001/api/tasks -H "Content-Type: application/json" -d '{"title":"画面を作る"}'
curl -X PATCH http://localhost:3001/api/tasks/2
curl -i http://localhost:3001/api/tasks/99
```

2. 削除のAPIを追加する

DELETE /api/tasks/:id

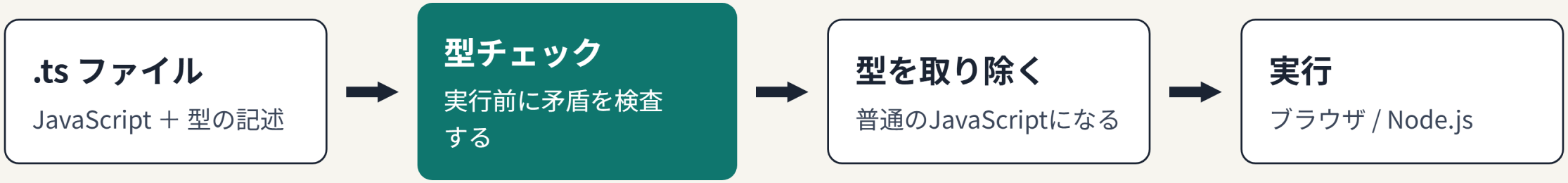
・成功時：204（ボディなし）

・存在しないID：404

ヒント

IDでタスクを探し、見つからなければ404を返す流れは、server.js の PATCH /api/tasks/:id と同じ。PATCH の実装を参考にする

TypeScript ＝ JavaScript ＋ 型の記述



コードが大きくなると、「この関数に何を渡せばよいか」がコードを読まないと分からなくなる。その情報を型として書き、補完とエラー検出に使う

現在のフロントエンド開発ではTypeScriptを使うのが標準。Java経験者にとっては、型のある書き方に近づく

Javaの型とは考え方が違う3つの点

1

名前ではなく「形」で判定

Java：同じクラスか、継承しているか
TS：必要なプロパティを持っているか

2

実行の前に型が消える

Java：実行中も、instanceof やキャストで型を検査
TS：実行前に JavaScript に変換され、そのとき型情報はすべて取り除かれる

3

null を入れられるかも型で表す

Java：どの参照型にも null が入る
TS：string に null や undefined は入らない。入れるなら型に明記

この3つを、順にコードで確かめていきます。

準備：task-api のTypeScriptの設定

```
# 実行済み（package.json の devDependencies に入っている）
npm install -D typescript tsx @types/node @types/express @types/cors
# typescript：型チェックを行う tsc コマンド
# tsx：.ts を JavaScript に変換してすぐ実行する（型チェックはしない）
# @types/...：JS製ライブラリに後付けで型情報を与える
```

tsconfig.json（作成済み）

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "strict": true,    // 最重要。無いと null / undefined の検査が行われない
    "noEmit": true,
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

```
npx tsc -p ts-samples          # ts-samples フォルダの型チェック
npx tsx ts-samples/erasure.ts   # 1ファイルを実行（型チェックはしない）
```

node/task-api/package.json
node/task-api/tsconfig.json

型は変数名の後ろに書く。省略もできる

```ts
const title: string = "設計書を読む";
const count: number = 3;
const done: boolean = false;
const titles: string[] = ["A", "B"];

// 初期値から型が分かる場合は省略できる（Javaの var と同じ）
const message = "こんにちは"; // string と推論される
// 関数は引数と戻り値に型を書く
function add(a: number, b: number): number {
  return a + b;
}

add(1, "2"); // エラー: Argument of type 'string' is not assignable to parameter of type 'number'.
```

TypeScript に対応したエディタ・IDEでは、保存前にエラーが表示される。変数にマウスを乗せると推論された型を確認できる

type は「形」を宣言する

```ts
type Task = {
  id: number;
  title: string;
  done: boolean;
  memo?: string; // ? を付けると省略可能（型は string | undefined になる）
};

const task: Task = { id: 1, title: "設計書を読む", done: true };
function label(task: Task): string {
  return `#${task.id} ${task.title}`;
}

// Task と宣言していないが、必要なプロパティをすべて持っているのでOK
const row = { id: 9, title: "DBから取得した行", done: false, createdBy: "sato" };
label(row);
```

new もコンストラクタも無い。{} で作ったオブジェクトの形が合っていれば、その型として扱われる

ユニオン型：「このどれか」を表す

```
type Status = "todo" | "doing" | "done";

function statusLabel(status: Status): string {
  switch (status) {
    case "todo": return "未着手";
    case "doing": return "作業中";
    case "done": return "完了";
  }
}

statusLabel("doing"); // OK
statusLabel("finish"); // エラー: 定義に無い文字列は渡せない
```

文字列そのものを型として並べると、Javaの **enum** に近い使い方ができる

string | undefined のように、「無いかもしれない」も | で表す

絞り込み：FalsyチェックをTypeScriptが理解する

```ts
const found = tasks.find((t) => t.id === 2); // 型は Task | undefined

console.log(found.title); // エラー: 'found' is possibly 'undefined'.

if (!found) {
  console.log("見つかりません");
} else {
  console.log(found.title); // OK。このブロック内では Task 型に絞り込まれている
}

console.log(found?.title ?? "（なし）"); // OK。?. と ?? もそのまま使える
```

Javaなら実行時に NullPointerException になるコードを、実行前に指摘する

第1部の **if (!x)・?.・??** を書けばエラーが消える。曖昧さを扱うイディオムが、安全性を証明する手段になる

型は実行前に取り除かれ、実行中には存在しない

```ts
// コンパイルエラーにも実行時エラーにもならない
const data = JSON.parse('{"id":"1","title":123}') as Task;

console.log(typeof data.id); // "string"（型の上では number のはず）
```

as は検査ではなく宣言
Javaのキャストは、失敗すると ClassCastException を投げる。as は何も検査しない

TypeScriptが保証するのは、自分たちが書いたコード同士の整合性だけ。**外から来るデータは、実行中に自分で確かめる**

型	意味	使いどころ
any	型チェックを諦める。何でもできてしまう	原則使わない
unknown	何が入っているか分からない。確かめるまで使えない	外から来たデータの受け取り

演習10 ・ TypeScript（40分）

server.js をTypeScript化する

次の手順で、server.js を TypeScript に書き換えてください。

1. server.js を src/index.ts にコピーし、npx tsc を実行する。エラーは何件出るか
2. tasks、find の結果、req.params.id、req.body にマウスを乗せ、推論された型をメモする
3. req.body?.title?.trim() を req.body.title.trim() に書き換え、ボディなしでPOSTする
4. 型を付けて仕上げる：Task 型を定義して tasks に型注釈を付ける。ID検索を findTask(id: string): Task | undefined にまとめる。req.body の title が文字列、done が真偽値であることを確かめてから使う
5. package.json の scripts に "dev": "tsx watch src/index.ts" を登録する

目的

型推論が「どこまで守ってくれて、どこから守ってくれないか」を自分の目で確かめる

node/task-api/server.js
node/task-api/src/index.ts（演習で作成する）

第3部 ・ まとめ

第3部でおさえた4つのこと

1. Node.jsはJavaScriptの実行環境。npmはMaven、package.json は pom.xml にあたる

2. 戻り値型が Promise の関数を呼ぶときは await を付ける。付け忘れると Promise そのものが入る

3. TypeScriptは形で型を判定し、undefined を入れられるかも型で表す。Falsyチェックがそのまま絞り込みになる

4. 型は実行前に取り除かれる。外から来るデータは、型ではなく実行中の検査で守る

JSONだけを返すサーバーができた。第4部では、このJSONから画面を組み立てる側を作る

第4部

React
Next.js

第3部のAPIを呼び出して、ブラウザ側で画面を組み立てる。最後にサーバー側の描画と組み合わせる

「どう変えるか」ではなく「どうなるか」を書く

第1部のDOM操作

変更の手順を書く。要素を探す、作る、追加する、件数表示も自分で書き換える

更新箇所が増えると漏れが起きる

React

「データがこうなら、画面はこうなる」という対応関係だけを書く

データを変えると、画面は自動で更新される

Thymeleaf（テンプレート＋データ→HTML）を、ブラウザ内でデータが変わるたびに行うもの、と考えると近い。

Viteで作った React + TypeScript のプロジェクトを動かす

```
cd /work/react-next/task-ui   # コンテナの中で実行する
npm install
npm run dev -- --host   # http://localhost:5173 で起動
```

ファイル	役割
index.html	唯一のHTML。中身は だけ
src/main.tsx	起点。App を #root の中に描画する
src/App.tsx	画面本体。この部ではここを書き換えていく
src/index.css	全体のCSS。第1部のCSSに置き換え済み

--host を忘れない：付けないと、PCのブラウザからコンテナ内のViteに接続できない

コンポーネント ＝ 画面の一部を返す関数

```tsx
function TaskItem({ task }: { task: Task }) {   // 引数（props）は分割代入＋型注釈で受け取る
  return (
    <li className={task.done ? "done" : ""}>
      <span>{task.title}</span>
      <button type="button">完了切替</button>
    </li>
  );
}
export default function App() {
  const tasks: Task[] = [ ... ];
  // task={task} が部品（TaskItem）への引数になる
  return (
    <main>
      <h1>タスク一覧</h1>
      <ul className="task-list">
        {tasks.map((task) => (<TaskItem key={task.id} task={task} />))}
      </ul>
    </main>
  );
}
```

JSX：JavaScriptの中にHTMLのような記述を書く。関数名（部品名）は大文字で始める

くり返しは第1部の **map**、条件は三項演算子。専用の構文は無い

react-next/task-ui/src/App.tsx（state を加える前の形）

Thymeleaf と JSX を見比べる

やりたいこと	Thymeleaf	JSX
値の表示	th:text="${task.title}"	{task.title}
くり返し	th:each="task : ${tasks}"	{tasks.map((task) => ...)}
条件付き表示	th:if="..."	{条件 && ... }
classの切り替え	th:classappend="..."	className={条件 ? "done" : ""}

JSXのルール

- { } の中にはJavaScriptの式を書ける
- class ではなく className と書く
- 返す要素は1つにまとめる
- くり返しの要素には key を付ける

Thymeleafは「HTMLの中に式を埋め込む」。
JSXは「JavaScriptの中にHTMLを書く」。
埋め込んだ文字列は自動でエスケープされる

第4部 ・ 4-2 注意

&& による条件付き表示とFalsyの落とし穴

```
// 0件のとき、画面に「0」と表示されてしまう
{tasks.length && <p>{tasks.length}件あります</p>}

// 正しい
{tasks.length > 0 && <p>{tasks.length}件あります</p>}
```

&& は左がFalsyなら、左の値そのものを返す。
Reactは false・null・undefined を表示しないが、**数値の 0 は表示する**

条件には、真偽値になる式を書く

やってみよう（10分） tasks に3件目を追加する／tasks を空配列にして表示を確認する／一覧の下に「未完了：1件」と表示する

state：変化するデータをReactに管理させる

```
const [tasks, setTasks] = useState<Task[]>([]);
//     ↑現在の値   ↑値を変更する関数              ↑初期値
```

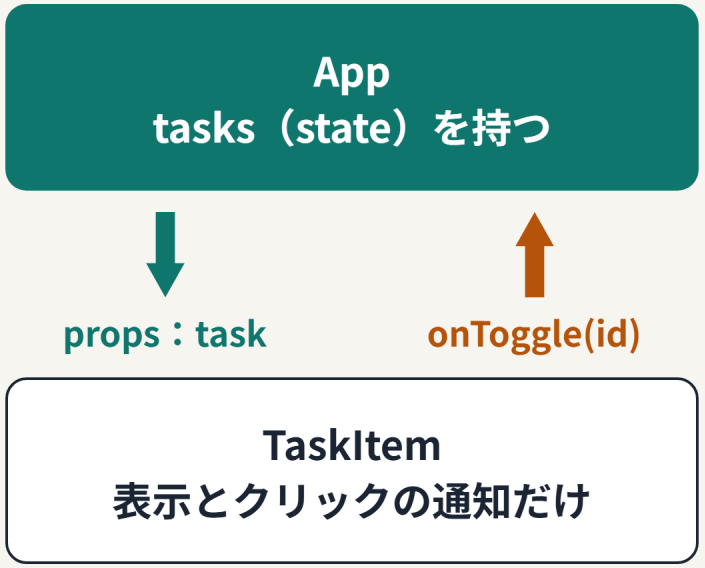
ルールは1つ：stateは直接書き換えず、
変更用の関数に「新しい値」を渡す

```
tasks.push(newTask); // 誤り：配列の中身は変わるが、Reactは変更に気づかない
setTasks([...tasks, newTask]); // 正しい：新しい配列を作って渡す
```

追加と完了切替をstateで実装する

```
const [tasks, setTasks] = useState<Task[]>([ ... ]);
const [title, setTitle] = useState("");

function addTask() {
  if (!title.trim()) return;
  const newTask: Task = { id: Date.now(), title: title.trim(), done: false };
  setTasks([...tasks, newTask]);
  setTitle("");
}

function toggleTask(id: number) {
  setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
}

const openCount = tasks.filter((t) => !t.done).length; // tasks が変われば自動で再計算される

// JSX（抜粋）：入力欄の値もstateで持つ
<input value={title} onChange={(event) => setTitle(event.target.value)} />
<TaskItem key={task.id} task={task} onToggle={toggleTask} />
```

データは親から子へ、イベントは子から親へ



件数表示を更新するコードが無い
tasks が変われば openCount も計算し直される

入力欄の値もstateで持つ
DOMから値を読み取るコードは登場しない

データを変更するのは親
子はクリックされたことを伝えるだけ。関数もpropsで渡せる

演習11 ・ state（15分）

stateを直接書き換えると、何が起きるか

次の手順で、stateの扱い方を確かめてください。

1. 追加と完了切替が動くこと、未完了の件数が連動することを確認する
2. toggleTask を、tasks.find(...) で見つけたタスクの done を直接書き換える形に変える。画面が更新されないことを確かめたら、元に戻す
3. 各タスクに「削除」ボタンを追加する

最初に1回だけ、APIから一覧を取得する

```
import { useEffect, useState } from "react";

const API = "http://localhost:3001/api/tasks";
export default function App() {
  const [tasks, setTasks] = useState<Task[]>([]); // 初期値を空にする

  useEffect(() => {
    async function load() {
      const res = await fetch(API);
      setTasks(await res.json());
    }
    load();
  }, []); // [] = 最初の表示時にだけ実行する
```

データの置き場所を、ブラウザのメモリから第3部のAPIに変える。JSXの部分は変更しない

react-next/task-ui/src/App.tsx（演習12で、この形に書き換える）

追加と切替：APIを呼び、結果でstateを更新する

```
async function addTask() {
  if (!title.trim()) { setMessage("タイトルは必須です"); return; }
  const res = await fetch(API, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title }),
  });
  const created: Task = await res.json();
  setTasks([...tasks, created]); // サーバーが採番したIDを持つタスクを追加
  setTitle("");
}
async function toggleTask(id: number) {
  const res = await fetch(`${API}/${id}`, { method: "PATCH" });
  const updated: Task = await res.json();
  setTasks(tasks.map((t) => (t.id === id ? updated : t)));
}
```

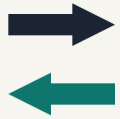
ページを再読み込みしても、追加したタスクが残る。データはAPIサーバー側にある

react-next/task-ui/src/App.tsx（演習12で、この形に書き換える）

CORS：ポートが違えば「別のサイト」

画面（Vite）
`http://localhost:5173`
ブラウザ内のJavaScriptが fetch する

別オリジンへのリクエスト



Access-Control-Allow-Origin ヘッダーで許可

API（Express）
`http://localhost:3001`

```
app.use(cors());
```

ブラウザは安全のため、別オリジンへの fetch の結果を、相手のサーバーが許可しない限りJavaScriptに渡さない

「画面からAPIが呼べない」という問い合わせの多くは、この設定が原因。Spring Bootでは @CrossOrigin などで設定する

演習12 ・ API連携（25分）

APIとつなぎ、ページのソースを見る

次の手順で、画面とAPIをつないでください。

1. src/App.tsx を、APIから一覧を取得し、追加・完了切替もAPIを呼ぶ形に書き換える。追加・完了切替・再読み込み後の保持を確認する
2. 第3部で作ったAPIサーバー（node/task-api/server.js）の app.use(cors()) をコメントアウトして画面を再読み込みし、Consoleのエラーを読む。確認したら元に戻す
3. ページのソースを表示（Ctrl+U）し、タスクのタイトルを探す
4. 前の演習で作った削除ボタン（deleteTask 関数）を、第3部で作った DELETE /api/tasks/:id を呼ぶように変更する

Next.js ＝ Reactのためのフレームワーク

CSR（Viteで作ったReactアプリ）の弱点

- 最初に届くHTMLが空。JSの読み込みとAPIの応答が終わるまで何も表示されない
- 検索エンジンなど、HTMLだけを読む相手に内容が伝わりにくい
- ページの切り替え（ルーティング）は別のライブラリが必要

Next.jsの解決

Reactのコンポーネントを、サーバー側でも実行できるようにする

ルーティングやビルドもまとめて提供する。Javaに対するSpring Bootのような位置づけ

第2部のテンプレートエンジン（サーバーでHTMLを作る）と、Reactを、1つの言語・1つの書き方で組み合わせる

ファイルの置き場所がURLになる

```
cd /work/react-next/task-next   # コンテナの中で実行する
npm install
npm run dev   # http://localhost:3000 で起動（--host は不要）
```

ファイル	役割
app/layout.tsx	全ページ共通の外枠（html や body）
app/page.tsx	/ のページ
app/tasks/[id]/page.tsx	/tasks/1、/tasks/2 などのページ。[id] がパラメータになる
app/loading.tsx	データ取得中に、ページの代わりに表示される内容

Spring Bootでは @GetMapping("/tasks") でURLを決めた。Next.jsでは app フォルダの構成がそのままURLになる。

サーバーコンポーネント：サーバー側で取得して描画

```
// app/page.tsx
import { API } from "./types";
import type { Task } from "./types";
export default async function Home() {   // 指定が無ければサーバー側で実行される。useEffect も useState も不要
  const res = await fetch(API);
  const tasks: Task[] = await res.json();
  return (
    <main>
      <h1>タスク一覧</h1>
      <ul className="task-list">
        {tasks.map((task) => <li key={task.id}><span>{task.title}</span></li>)}
      </ul>
    </main>
  );
}

// app/loading.tsx
export default function Loading() {
  return <p>読み込み中...</p>;
}
```

loading.tsx は省略できない：現在の初期設定では、無いとエラーになる

react-next/task-next/app/page.tsx（TaskList を使う前の形）
react-next/task-next/app/loading.tsx

ブラウザで動かす部分だけ "use client" で分ける

観点	サーバーコンポーネント（既定）	クライアントコンポーネント（"use client"）
実行場所	サーバー	ブラウザ（最初の表示分はサーバーでも実行）
できること	データ取得、DBやファイルへのアクセス、秘密情報の利用	useState、onClick などのイベント処理
できないこと	state、イベント処理	サーバーだけが持つ情報へのアクセス
近いもの	第2部のThymeleafテンプレート	第2部の tasks.js、Viteで作ったReactアプリ

構造は第2部と同じ。サーバーからブラウザへのデータの受け渡し（第2部では data-id）が、propsで済む。

クライアントコンポーネント：app/TaskList.tsx

```
"use client";

export default function TaskList({ initialTasks }: { initialTasks: Task[] }) {
  const [tasks, setTasks] = useState(initialTasks);
  async function toggleTask(id: number) {
    const res = await fetch(`${API}/${id}`, { method: "PATCH" });
    const updated: Task = await res.json();
    setTasks(tasks.map((t) => (t.id === id ? updated : t)));
  }
  return (
    <ul className="task-list">
      {tasks.map((task) => (
        <li key={task.id} className={task.done ? "done" : ""}>
          <span><Link href={`/tasks/${task.id}`}>{task.title}</Link></span>
          <button type="button" onClick={() => toggleTask(task.id)}>完了切替</button>
        </li>
      ))}
    </ul>
  );
}
```

動的ルート：app/tasks/[id]/page.tsx

```
type Props = {
  params: Promise<{ id: string }>;
};
export default async function TaskDetail({ params }: Props) {
  const { id } = await params;   // URLの [id] の部分。/tasks/1 なら "1"（文字列）。@PathVariable と同じ役割
  const res = await fetch(`${API}/${id}`);
  if (!res.ok) {
    notFound();   // 404ページを表示する
  }
  const task: Task = await res.json();
  return (
    <main>
      <h1>タスク #{task.id}</h1>
      <p>状態：{task.done ? "完了" : "未完了"}</p>
      <Link href="/">一覧へ戻る</Link>
    </main>
  );
}
```

フォルダ名の **[id]** は、角括弧も含めてそのまま入力する

このページもサーバー側でデータを取得する。app/loading.tsx を **app/tasks/[id]/loading.tsx** にコピーする

演習13 ・ Next.js（30分）

「このコードはどこで動いているか」を確かめる

次の手順で、Next.js の動きを確かめてください。

1. 一覧の表示、完了切替、詳細表示、「一覧へ戻る」が動くことを確認する
2. http://localhost:3000/tasks/99 を開き、404ページが表示されることを確認する
3. app/page.tsx（サーバーコンポーネント）の h1 に onClick を付け、どんなエラーが出るかを読む。確認したら元に戻す
4. app/page.tsx と app/TaskList.tsx のそれぞれに console.log("実行されました") を書き、ターミナルとブラウザのConsoleのどちらに出力されるかを確かめる
5. 時間があれば、Viteで作った App.tsx の追加フォーム（入力欄・追加ボタン・addTask 関数）を TaskList.tsx に移植する

手順4のポイント

Next.jsで迷ったときに、最初に調べること

`react-next/task-next/app/page.tsx`
`react-next/task-next/app/TaskList.tsx`

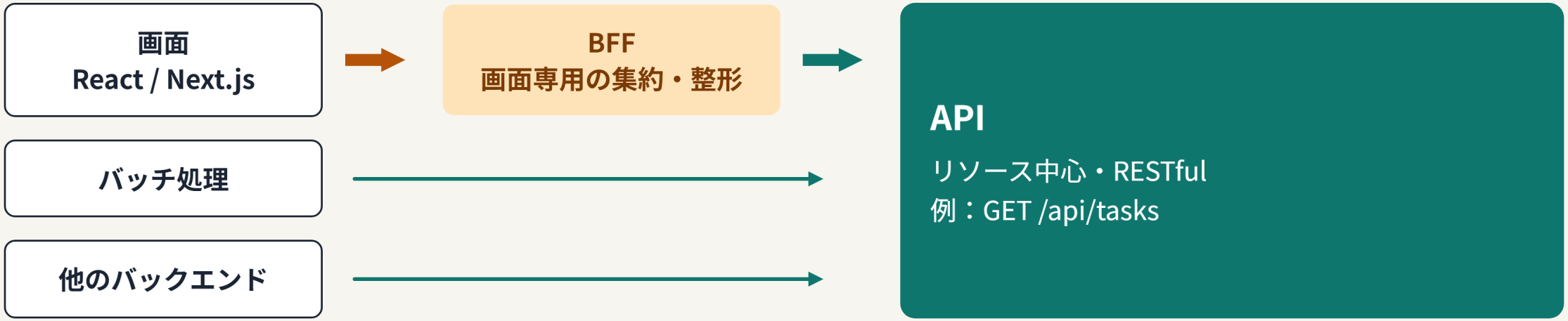
第4部 ・ 4-6 まとめ

同じ画面を3通りの方式で作った

観点	テンプレートエンジン（第2部）	API＋CSR（第3部・第4部前半）	Next.js（第4部後半）
HTMLを組み立てる場所	サーバー	ブラウザ	サーバーとブラウザの両方
サーバーが返すもの	完成したHTML	空のHTMLとJS、データはJSON	内容の入ったHTMLとJS
ソースにデータがあるか	ある	ない	ある
操作時の画面更新	ページ全体を再読み込み	変わった部分だけ	変わった部分だけ
使う言語	Java＋HTML＋少量のJS	サーバーは任意、画面はTS	TypeScript
向いている場面	社内向け管理画面、フォーム中心の業務システム	操作が多い画面、スマホアプリとAPIを共用	表示速度や検索流入が重要な公開サイト

新しい方式が常に優れているわけではない。配属先がどの方式かを見分け、データの流れを説明できることがゴール

APIは画面の都合ではなく、リソースを中心に設計する



API：呼び出し元を選ばない。特定の画面に合わせて項目を足すと、利用者ごとの都合に振り回され保守しにくくなる

BFF：「一覧に必要な項目を1回で取りたい」といった画面の都合を吸収する。Next.jsのサーバー側が担える

第4部 ・ 4-6 まとめ

バックエンドエンジニアとして持ち帰ること

APIはリソースを中心に設計する

呼ぶのは画面だけではない。画面ごとの都合はAPIに持ち込まず、BFFで吸収する

画面から来るデータは信用しない

画面側の入力チェックもTypeScriptの型も、サーバーを守らない

CORSとステータスは画面に直結する

画面側は res.ok で分岐する。エラー時に200を返すAPIは画面側を困らせる

Networkタブが共通言語になる

画面とAPIのどちらに原因があるかは、リクエストとレスポンスを見れば切り分けられる

おわりに

この研修で扱わなかったこと

分野	必要になったときに調べるキーワード
CSS	レスポンシブデザイン、Grid、Tailwind CSS
JavaScript	クラス構文、クロージャ、this の挙動
TypeScript	ジェネリクスの定義、ユーティリティ型、スキーマ検証ライブラリ
React	useEffect の依存配列、カスタムフック、状態管理・データ取得ライブラリ
Next.js	キャッシュ、Server Actions、認証、デプロイ
周辺	テスト、Lintとフォーマッタ

調べる起点は、各技術の公式ドキュメントから